Równania nieliniowe - metoda Newtona, bisekcji oraz schematy iteracyjne

Autor: Jacek Łoboda, Jakub Staniszewski
Data: 05.05.2026

1. Wprowadzenie

Celem niniejszego laboratorium było zapoznanie się z iteracyjnymi metodami rozwiązywania równań nieliniowych. W trakcie zajęć skupiono się na badaniu warunków oraz szybkości zbieżności (wyznaczając jej rząd), a także analizie ograniczeń popularnych algorytmów. Kluczowym elementem zadania było zdiagnozowanie przypadków, w których szybko zbieżna metoda Newtona napotyka problemy i ulega awarii, co zmusza do stosowania innych technik – na przykład metody bisekcji.

2. Analiza schematów iteracyjnych

Rozważano równanie $f(x) = x^2 - 3x + 2 = 0$ z pierwiastkiem $\alpha = 2$.

Analiza teoretyczna zbieżności

Wartości bezwzględne pochodnych funkcji iterujących w punkcie $\alpha = 2$:

1. $|\phi_1'(2)| = |4/3| \approx 1.33$ (warunek $|\phi'(\alpha)| < 1$ niespełniony $\rightarrow$ rozbieżność)
2. $|\phi_2'(2)| = |3/4| = 0.75$ (zbieżność liniowa, $p = 1$)
3. $|\phi_3'(2)| = |1/2| = 0.50$ (zbieżność liniowa, $p = 1$)
4. $|\phi_4'(2)| = |0| = 0$ (zbieżność kwadratowa, $p = 2$)

Analiza wyników empirycznych

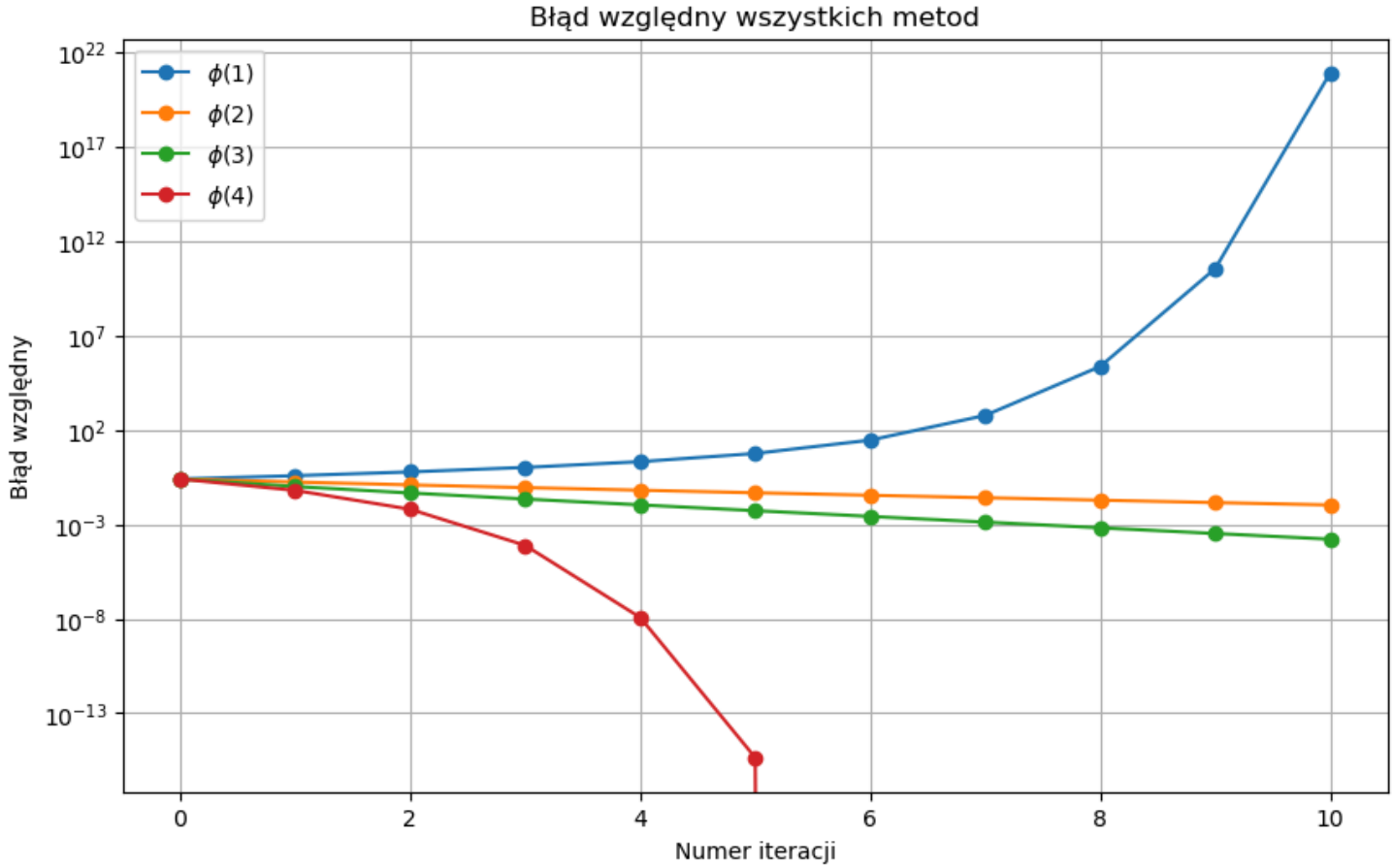
Zestawienie danych w Tabeli 1 oraz na wykresach potwierdza powyższe przewidywania:

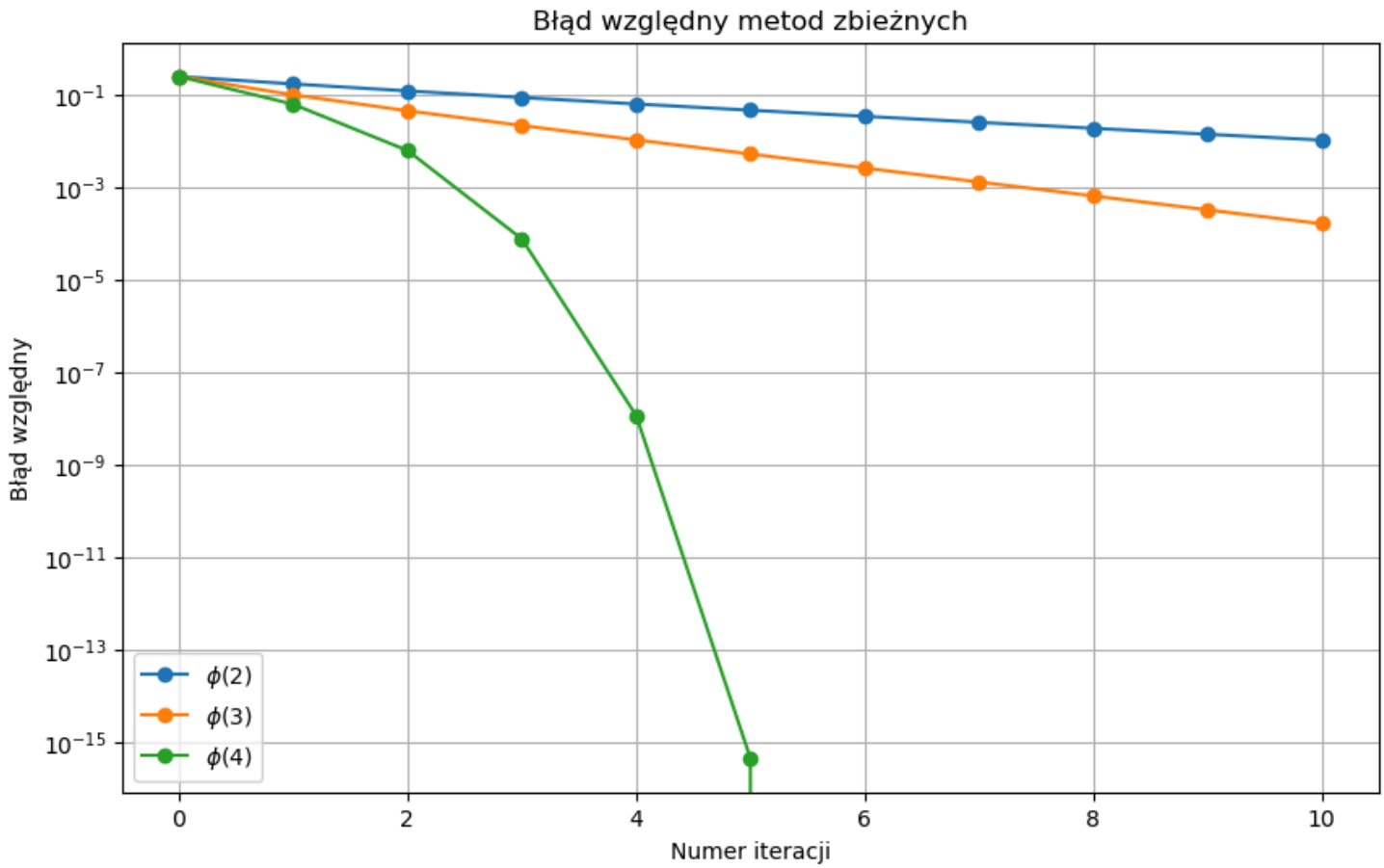
- Schemat 1: Zgodnie z teorią wartości r_k nie dążą do stabilnej wartości, a błąd rośnie, co potwierdza rozbieżny charakter punktu stałego.
- Schemat 2 i 3: Obie metody wykazują zbieżność liniową. Schemat 3 zbiega szybciej, co wynika z mniejszej wartości stałej błędu.

- Schemat 4: Wykazuje zbieżność kwadratową ($r_k \approx 2$). Metoda szybko redukuje błąd, osiągając granicę precyzji maszynowej w 5. iteracji. Wartości NaN pojawiają się, gdy błąd kolejnego kroku staje się równy zero w ramach precyzji obliczeniowej, co uniemożliwia dalsze wyznaczanie rzędu.

Tabela 1. Wyznaczone rzędy zbieżności r_k

Iteracja	r_{ϕ_1}	r_{ϕ_2}	r_{ϕ_3}	r_{ϕ_4}
1	1.1333	0.9363	0.8604	1.6609
2	1.1917	0.9529	0.9354	1.9138
3	1.2818	0.9651	0.9688	1.9944
4	1.4224	0.9740	0.9847	1.9432
5	1.6280	0.9806	0.9924	NaN
6	1.8544	0.9855	0.9962	NaN
7	1.9787	0.9892	0.9981	NaN
8	1.9994	0.9919	0.9990	NaN
9	1.9999	0.9939	0.9995	NaN


Błąd względny wszystkich metod
$\phi(1)$
$\phi(2)$
$\phi(3)$
$\phi(4)$
Błąd względny
Numer iteracji


Błąd względny metod zbieżnych
$\phi(2)$
$\phi(3)$
$\phi(4)$
Błąd względny
Numer iteracji

3. Słabe punkty metody Newtona

W kolejnym etapie badano działanie standardowej funkcji bibliotecznej `scipy.optimize.newton`. Wyznaczono 4 punkty startowe dla podanych funkcji, przy których metoda ulega awarii:

- $f(x) = x^3 - 5x$, dla $x_0 = 1$
- $f(x) = x^3 - 3x + 1$, dla $x_0 = 1$
- $f(x) = 2 - x^5$, dla $x_0 = 0.01$
- $f(x) = x^4 - 4.29x^2 - 5.29$, dla $x_0 = 0.8$

Przyczyny niepowodzeń:

1. **Pochodna równa zeru:** (np. dla drugiego przypadku $f'(1) = 0$). Skutkuje to błędem dzielenia przez zero we wzorze na kolejny krok algorytmu ($x_{n+1} = x_n - f(x_n)/f'(x_n)$).
2. **Wpadanie w cykl oscylacji:** Dla $x^3 - 5x$ w punkcie 1 iteracje przeskakują symetrycznie z jednej strony zera na drugą, wpadając w nieskończoną pętlę i nie zbliżając się do pierwiastka.
3. **Płaskie otoczenie funkcji:** (przypadek z małym x_0). Bardzo niska wartość pochodnej sprawia, że styczna poprowadzona w punkcie startowym przecina oś w wartości ekstremalnie dużej. Iteracja "wybiega" poza zakres rozsądnych wartości liczbowych, rzucając błędem biblioteki i całkowicie opuszczając otoczenie szukanego miejsca zerowego.

Rozwiązanie problemu:
We wszystkich "trudnych" przypadkach ratunkiem okazała się **metoda bisekcji**. Choć dużo wolniejsza (zbieżność liniowa), nie zależy ona od wartości pochodnych. Jej wymogiem jest jedynie wskazanie przedziału ciągłości $[a, b]$, na krańcach którego funkcja ma przeciwne znaki.

4. Optymalizacja działań arytmetycznych

Ostatnie zadanie dotyczyło wykorzystania założeń metody Newtona do wykonywania działań sprzętowych tam, gdzie procesor posiada ograniczony zasób instrukcji:

1. **Odwrotność** $1/c$ (brak dzielenia): wyprowadzony wzór to $x_{n+1} = x_n(2 - c \cdot x_n)$.
2. **Odwrotność pierwiastka** $1/\sqrt{c}$ (tylko +, *, dzielenie przez 2 - bitshift): $x_{n+1} = \frac{x_n}{2}(3 - c \cdot x_n^2)$.
3. **Pierwiastek** $\sqrt{c}$**:** Choć schemat Herona radzi sobie dobrze, wymaga ogólnego dzielenia. By go uniknąć, możemy skorzystać ze wzoru z punktu 2, a uzyskany wynik ($1/\sqrt{c}$) po prostu pomnożyć operacją sprzętową przez c. W ten sposób otrzymujemy sam pierwiastek optymalnym kosztem, unikając kosztownego w taktowaniu dzielenia "ogólnego".

5. Układ równań nieliniowych

Ostatnim etapem było rozwiązanie układu równań:

1. $x_1^2 + x_2^2 = 1$
2. $x_1^2 - x_2 = 0$

Metodyka:
Zastosowano wielowymiarową metodę Newtona, korzystając z macierzy Jacobiego. Wyznaczono pochodne czcoąstkowe funkcji składowych, co pozwoliło na iteracyjne wyliczanie poprawki Δx poprzez rozwiązywanie układu równań liniowych $J \cdot \Delta x = -F$ w każdym kroku.

Wyniki:
Metoda Newtona dla układów równań wykazała się bardzo szybką zbieżnością.

- **Wyznaczone rozwiązanie:** $x_1 \approx 0.78615, x_2 \approx 0.61803$
- **Błąd względny:** Rzędu 10^{-16} (po 10 iteracjach), co oznacza osiągnięcie pełnej precyzji maszynowej.

Warto zauważyć, że dobór punktu startowego ma kluczowe znaczenie – przy punkcie $(0.5, 0.5)$ algorytm zbiegł do rozwiązania dodatniego, natomiast przy zmianie znaku x_1 w punkcie startowym, zbiegłby do symetrycznego rozwiązania ujemnego.

6. Podsumowanie

Dzisiejsze laboratorium świetnie ukazało dualizm algorytmów numerycznych rozwiązywania równań nieliniowych. Metoda Newtona (oraz ogólnie metody o wyższym rzędzie zbieżności) pozwala osiągnąć doskonałe rezultaty w zaledwie ułamek czasu działania innych algorytmów. Niestety, wymaga ona spełnienia rygorystycznych założeń (istnienie ciągłej 1. i 2. pochodnej, brak maksimów i minimów, dobry punkt startowy w sąsiedztwie rozwiązania).

W związku z tym praktyka programistyczna często dyktuje używanie rozwiązań hybrydowych: rozpoczynamy od metody niezawodnej, lecz powolnej (takiej jak bisekcja), by wejść w "bezpieczne" otoczenie lokalne pierwiastka. Gdy znajdziemy się odpowiednio blisko, przełączamy na szybko zbieżną metodę iteracyjną (jak Newton), znacząco redukując sumaryczny czas i koszt obliczeń.